NutriVision

AI-Powered Indian Food Nutrition Estimator

Final Technical Report · Milestones 6

Group	Group 2 — Data Science & AI Lab
Institute	IIT Madras
Project	NutriVision – AI Food Analyzer
Members	Sahil Raj · Sahil Sharma · Aman Mani Tiwari · Samreen Fathima · Tasneem Shahnawaz

1. Introduction & Problem Statement

NutriVision is an end-to-end AI system that accepts a photograph of a meal — including complex Indian multi-dish plates — and returns a detailed nutritional breakdown with personalised dietary suggestions. The system is specifically designed to address the limitations of existing nutrition tracking tools, which require manual food entry, assume fixed 100 g portions, and have poor coverage of Indian cuisines.

The core technical challenge is two-fold: (1) accurately identifying multiple food items in a single photograph, and (2) estimating the real-world portion weight of each identified item from 2-D pixel data alone, without depth sensors or specialised hardware. Downstream of these two problems, nutritional values are retrieved from a structured database and personalised recommendations are generated using a large-language-model pipeline grounded in ICMR dietary guidelines.

1.1 Objectives

- Develop a multi-food detection/classification system for Indian cuisine.
- Estimate portion size from a single photograph using geometry and density priors.
- Map detected foods to a structured nutritional database and compute macro/micronutrient intake.
- Perform personalised nutritional gap analysis against ICMR Recommended Dietary Allowances.
- Generate dietary suggestions via a RAG-augmented LLM pipeline (Gemini + MedGemma).
- Integrate all components into a deployable end-to-end application.

1.2 Comparison with Existing Solutions

The table below summarises how NutriVision addresses gaps in existing tools.

Feature	Traditional Apps	Basic Classifiers	NutriVision
Manual food entry required	Yes	No	No
Multi-food detection in one image	No	Limited	Yes (SAM3 + Classifier)
Portion size estimation from image	No	No (100 g fixed)	Yes (PCA + density)
Indian multi-dish meal support	Limited	Limited	Full support

Personalised nutrient gap analysis	Limited	No	Yes (ICMR RDA)
Conversational AI dietary suggestions	No	No	Yes (Gemini + MedGemma)

2. Datasets

NutriVision used two distinct data tracks aligned to the evolution of its pipeline: a detection track (early milestones) and a classification track (final pipeline). A custom nutrition database and a small in-house weight-estimation dataset supplemented both tracks.

2.1 Food Classification Dataset — Khana Dataset

The Khana dataset is a large-scale benchmark specifically designed for Indian food recognition. It formed the primary training corpus for the final classification backbone.

Attribute	Detail
Total images	131,819 validated food images
Food classes	80 Indian food categories
Train / Val / Test split	80% / 10% / 10% stratified (seed = 42)
Class imbalance	Min 12 images / class; Max 788 images / class; Avg ~164.8
Image diversity	Variable lighting, camera angle, plating style, backgrounds
Purpose	Training and evaluation of EfficientNetV2-S and ConvNeXtV2-Tiny classifiers

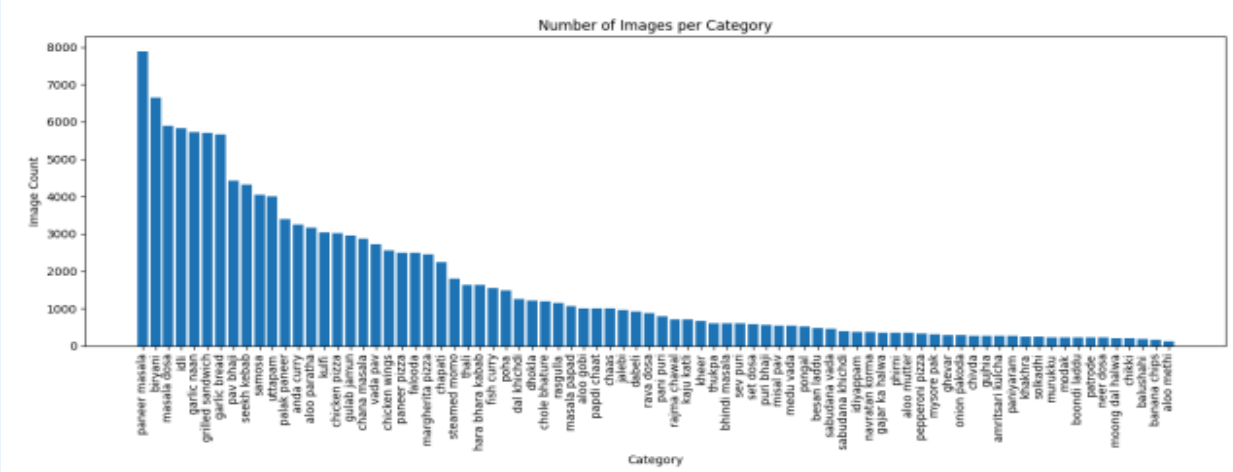


Figure 1: *Khana Dataset — class distribution bar chart (80 food categories)*

2.2 Detection Datasets (Earlier Pipeline — Milestones 2 & 3)

Three datasets were merged to train the YOLO-based detection pipeline in early milestones:

Dataset	Images	Classes	Key Features
IndianFood10 (Roboflow)	11,547	10	842 multi-dish plate images; YOLO segmentation masks
IndianFoodNet-30 (Roboflow)	5,446	30	Biryani, dosa, butter chicken; bounding box format
South Indian Food Detection	9,338	~31	Geographic balance for south-Indian cuisine
Indian Food YOLOv5 (M3 addition)	~14,000	81	Region-specific dishes; merged to 120 unified classes
TOTAL (merged, M3)	**39,185**	**118**	After removing classes with <50 annotations

2.3 Nutrition Database

The Indian Food Nutritional Values dataset (Kaggle 2025) contains per-100g macro and micronutrient data for 310+ Indian dishes (calories, protein, carbs, fat, fiber, sodium, iron). Missing values were imputed with the class median. This static CSV underpins the nutrition retrieval stage throughout all milestones.

2.4 Weight Estimation Dataset (Custom)

A small dataset of approximately 88 overhead plate photographs was collected by the team using a phone camera and digital kitchen scale. Each image was labelled with food name, pixel area, pixel ratio (food/plate), and actual weight in grams. After quality filtering, 63 usable records remained. This data was used to train an MLP regression model in the early pipeline before being superseded by the PCA + density estimation approach in the final pipeline.

3. Data Preprocessing & Exploratory Data Analysis

3.1 Detection Dataset Preprocessing (Milestones 2 & 3)

Merging three datasets with different naming conventions required a canonical label-mapping strategy:

- Spelling normalisation: e.g. 'briyani' → 'biryani', 'Idli' → 'idli'.

- Format normalisation: e.g. 'PalakPaneer' → 'palak_paneer'.
- Regional synonyms collapsed: e.g. 'satham' → 'white_rice'.
- Class-index remapping: after sorting unified labels, each dataset's integer class IDs were re-mapped to the new unified index space.
- Low-frequency class removal: classes with fewer than 50 annotations were dropped, reducing the class count from 120 to 118.
- All images standardised to 640×640 pixels for YOLO training.

```
/content/nutrivision_merged/
│
├── data.yaml                    ← YOLO config
│
├── train/
│   ├── images/
│   │   ├── ds1_img1.jpg
│   │   ├── ds2_img45.png
│   │   └── ...
│   │
│   ├── labels/
│   │   ├── ds1_img1.txt
│   │   ├── ds2_img45.txt
│   │   └── ...
│
├── valid/
│   ├── images/
│   ├── labels/
│
├── test/
│   ├── images/
│   ├── labels/
```

Figure 2: *YOLO directory structure after merging all three datasets*

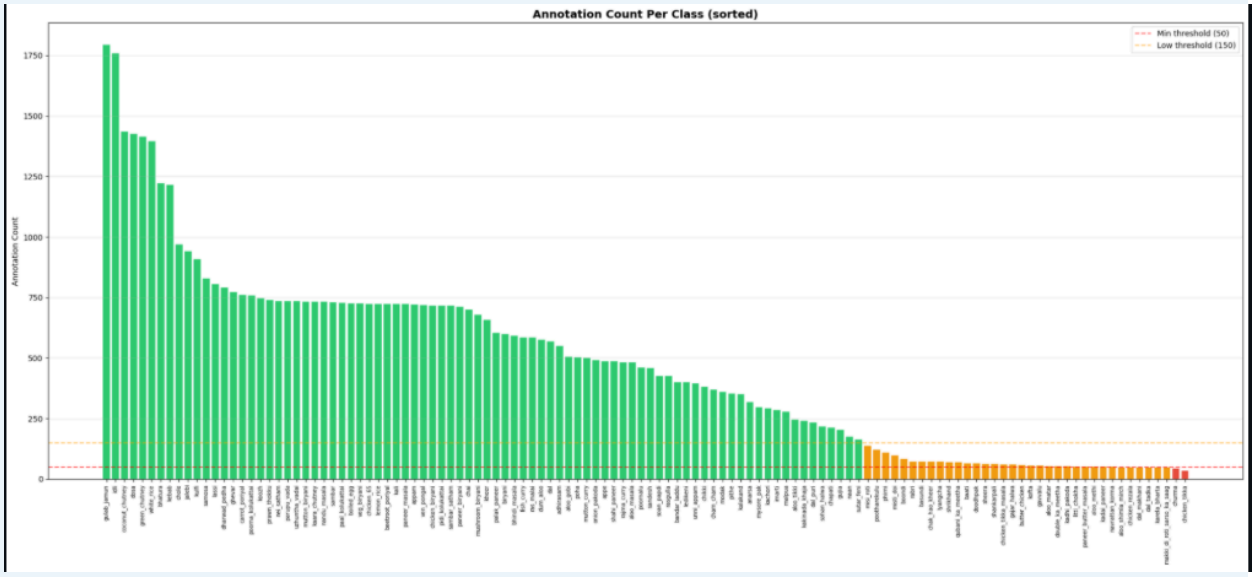


Figure 3: *Class distribution plot — 118 unified food classes with annotation counts*

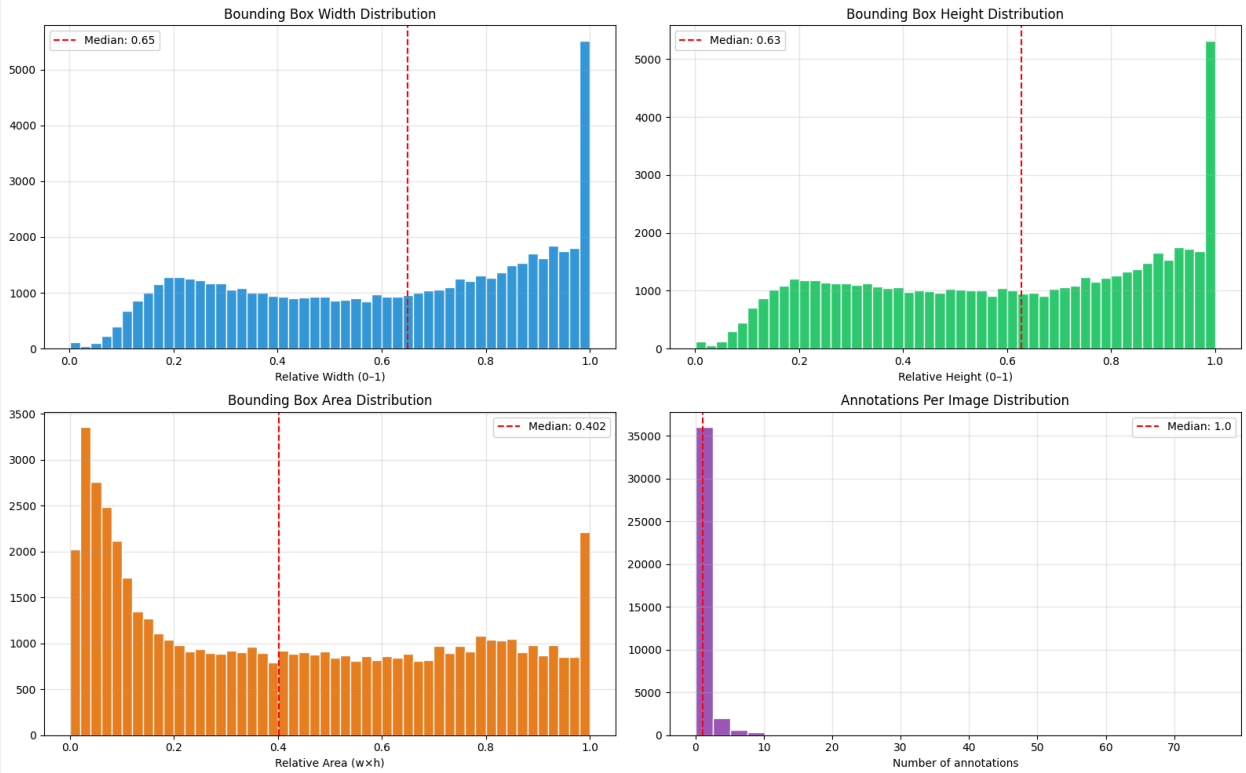


Figure 4: *Bounding box visualisation quality check on merged dataset*

3.2 Classification Dataset Preprocessing (Milestones 4 & 5)

For the Khana dataset, preprocessing was model-specific to match each architecture's training configuration:

Step	EfficientNetV2-S	ConvNeXtV2-Tiny
Resize	321 px → 300×300 centre crop	200 px → 176×176 centre crop
Normalisation (mean)	[0.5, 0.5, 0.5]	[0.485, 0.456, 0.406] (ImageNet)
Normalisation (std)	[0.5, 0.5, 0.5]	[0.229, 0.224, 0.225] (ImageNet)
Training augmentation	MixUp, CutMix, RandomResizedCrop, horizontal flip	RandomResizedCrop (176), horizontal flip, albumentations medium strength
Evaluation augmentation	None — centre crop only	None — centre crop only
Class imbalance handling	Loss-based weight adjustment	Weighted random sampler

Note: using the wrong image size or normalisation statistics during evaluation causes severe accuracy degradation — this was identified as the root cause of the initially poor ConvNeXtV2-Tiny results in Milestone 5, which were corrected by enforcing per-model preprocessing.

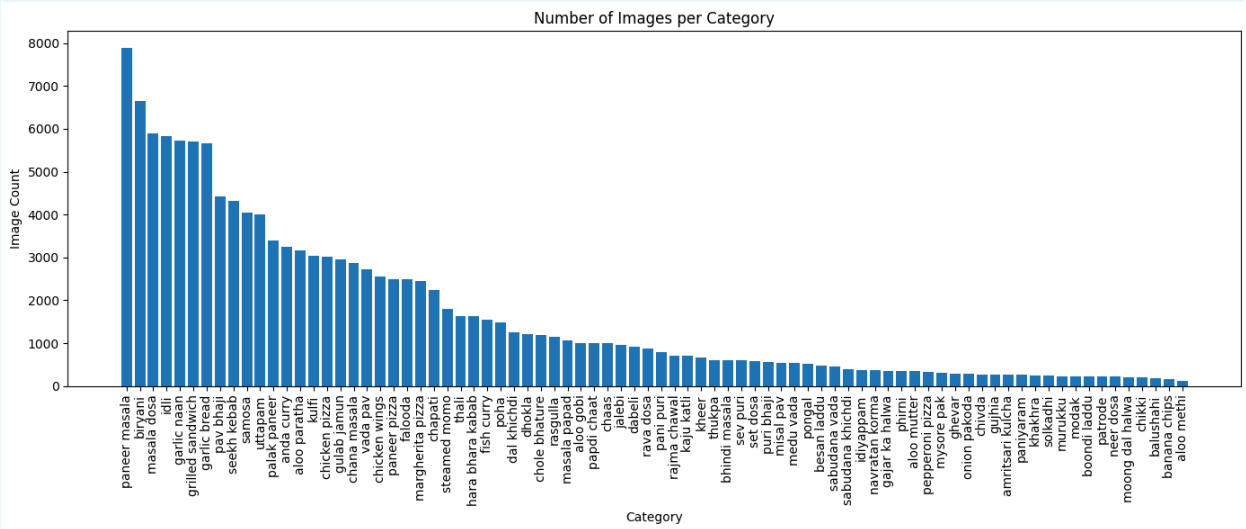


Figure 5: *Khana dataset EDA — image samples per class*

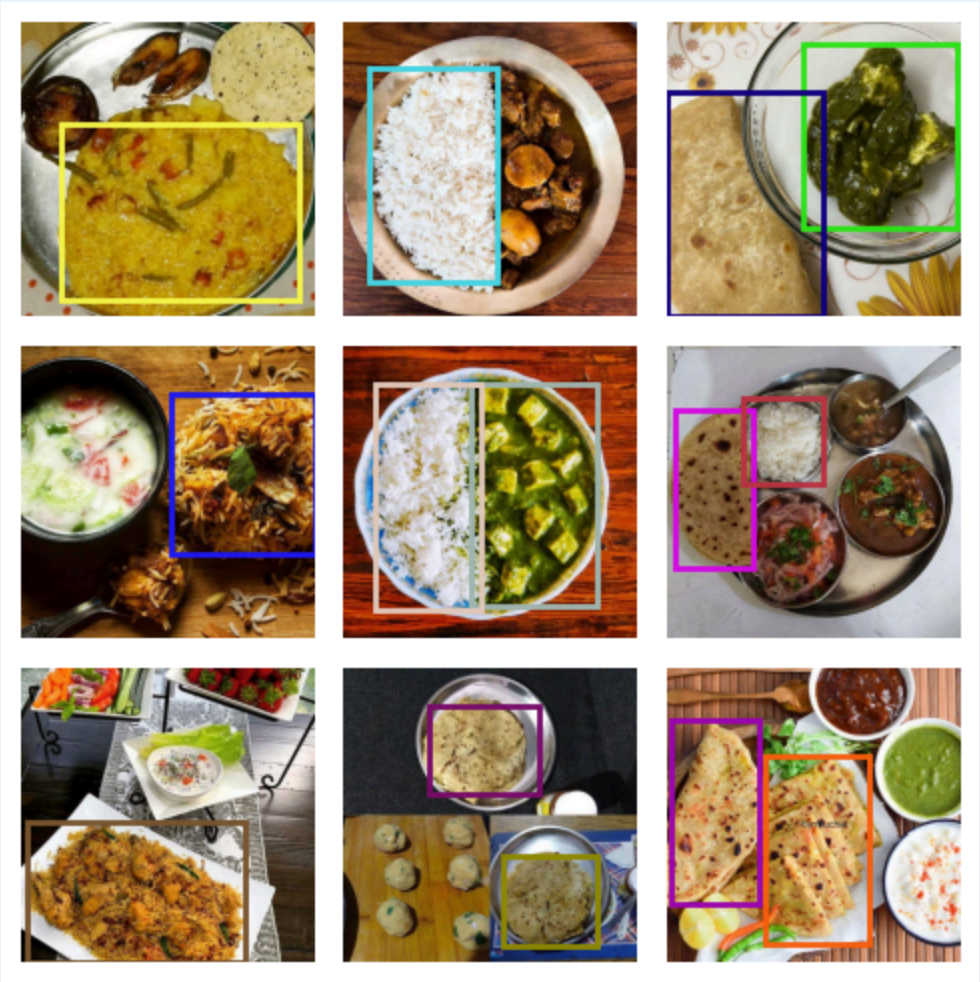


Figure 6: *Image dataset*

3.3 Nutrition Dataset EDA

The nutrition dataset was analysed across 310+ Indian dishes. Key findings: calorie distribution is right-skewed (most foods 50–250 kcal/100g); deep-fried snacks peak at 372 kcal; liquid dishes are lowest at 40–130 kcal. Macronutrient correlations confirmed fat is the dominant calorie driver, with protein weakly correlated to calories.

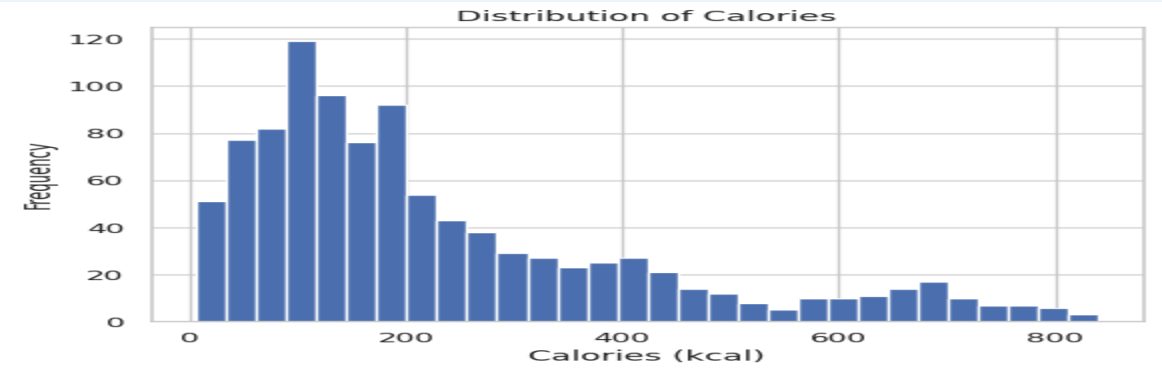
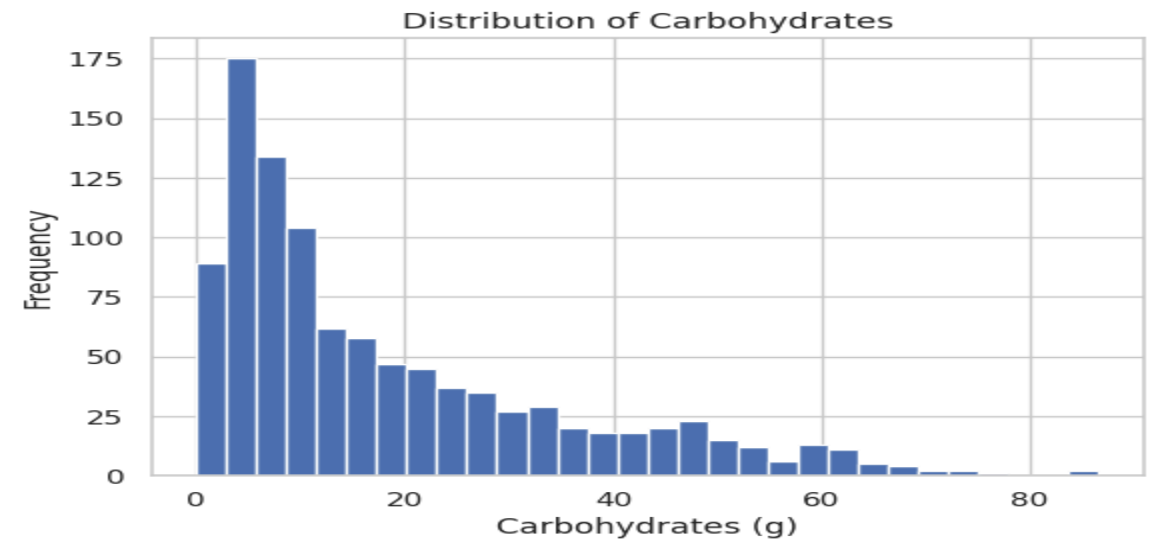
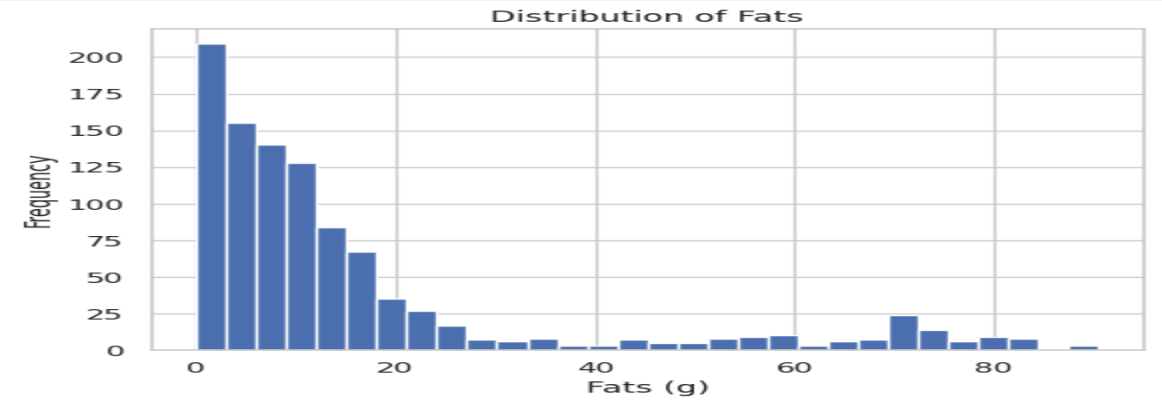


Figure 7: *Calorie distribution histogram across 310+ Indian foods*





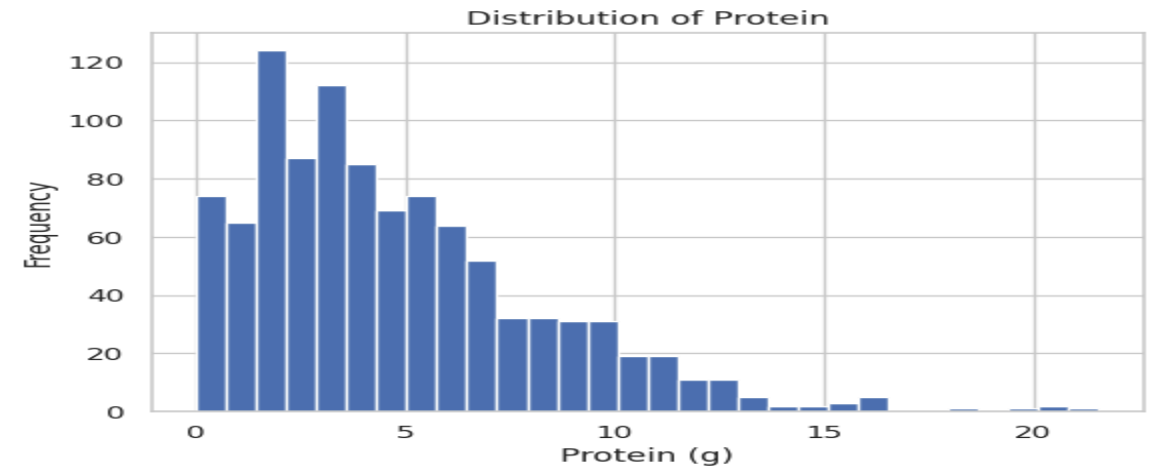


Figure 8: *Macronutrient profile by food category (protein/carbs/fat bar chart)* ▸ Insert from: **Nutrition.ipynb**
[Category macronutrient comparison cell]

4. Pipeline Evolution — From Detection to Segmentation-First

The NutriVision pipeline underwent a significant architectural evolution across milestones. Understanding this evolution is essential to understanding the final design choices.

4.1 Original Pipeline (Milestones 1 – 3)

Stage	Component	Description
1 — Detection	YOLOv8 / YOLOv12 / YOLOv26	Detect food items + bounding boxes
2 — Segmentation	SAM-2 (prompted by YOLO boxes)	Generate pixel masks per bounding box
3 — Weight Estimation	MLP regression model	pixel_ratio + pixel_food + class_encoded → weight (g)
4 — Nutrition Lookup	CSV + RAG (Pinecone)	Scale per-100g values by estimated weight
5 — Suggestions	Gemini API + MedGemma	RAG-grounded dietary recommendations

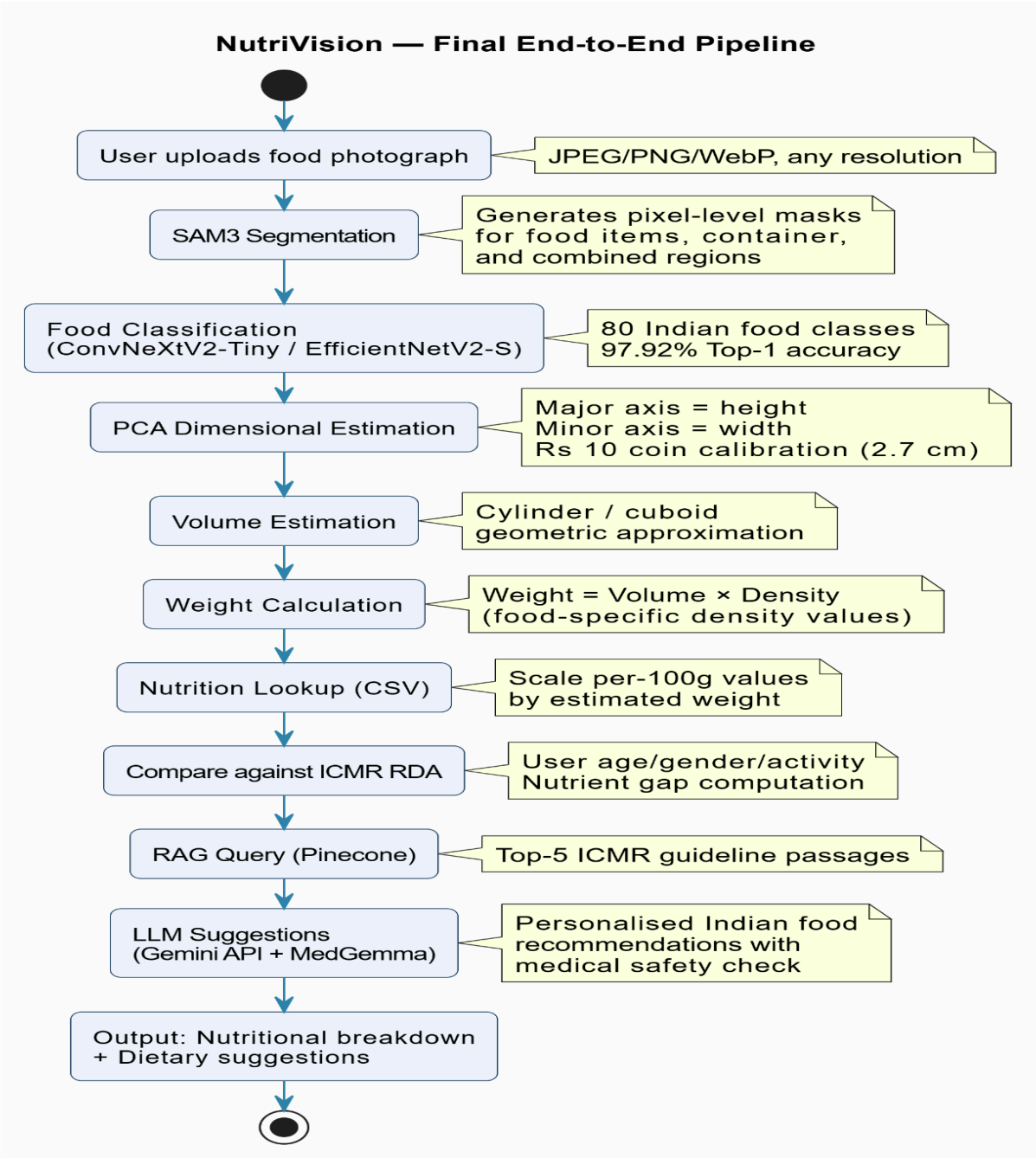
4.2 Revised Final Pipeline (Milestones 4 – 5)

The detection-first approach was replaced with a segmentation-first architecture using SAM3, and a dedicated classification model replaced the YOLO classification head. Weight estimation was redesigned from MLP regression to PCA-based geometric estimation with coin calibration.

Stage	Component	Description
1 — Segmentation	SAM3 (segment anything)	Direct pixel masks — food, container, food+container regions
2 — Classification	ConvNeXtV2-Tiny / EfficientNetV2-S	Classify each segmented food region into 80 Indian food classes
3 — Dimensional Estimation	PCA + coin calibration	Extract height/width from mask pixels; calibrate with Rs 10 coin (2.7 cm)
4 — Volume & Weight	Geometric approximation + density	Cylinder/cuboid approximation; multiply by food-specific density
5 — Nutrition Lookup	Structured CSV	Scale per-100g nutrient values by estimated weight

6 — Recommendations	MedGemma + Gemini API	ICMR RDA-grounded personalised dietary suggestions

4.3 Key Reasons for Pipeline Redesign



- SAM3 eliminates dependency on accurate bounding boxes and handles overlapping multi-dish scenarios more naturally.
- Dedicated classification models (ConvNeXtV2, EfficientNetV2) significantly outperform YOLO classification heads on fine-grained food categories.
- PCA-based weight estimation is more interpretable and does not require the large labelled regression dataset that the MLP approach demanded.
- Coin calibration provides a practical pixel-to-cm scale factor without requiring depth sensors.

5. Model Architecture & Training

5.1 Food Classification Models

Two deep learning architectures were selected as the primary food classifiers. Both were fine-tuned on the Khana dataset's 80 Indian food classes using pretrained ImageNet weights.

5.1.1 ConvNeXtV2-Tiny

ConvNeXtV2-Tiny is a modernised pure-convolutional network incorporating FCMAE (Fully Convolutional Masked AutoEncoder) pretraining. It was identified as the strongest single model in the project.

Parameter	Value
Parameters	27.9 M
Input image size	176 × 176 px
Batch size	128 (baseline); 256 via gradient accumulation (best)
Epochs	5 (baseline); 1 fine-tuning epoch with best hyperparameters
Optimiser	AdamW — Head LR: 1e-3, Backbone LR: 1e-4, Weight decay: 1e-2
Loss function	Cross-Entropy with label smoothing ε = 0.1
LR scheduler	1-epoch warmup → cosine decay
Regularisation	Dropout 0.2, weight decay, early stopping (patience=3)
Mixed precision	Yes (torch.amp.autocast)
Normalisation	ImageNet mean/std [0.485,0.456,0.406] / [0.229,0.224,0.225]
Test Top-1 Accuracy	97.92% (corrected evaluation)
Test Macro F1	0.9746
Inference throughput	150.7 images/sec (Tesla T4)

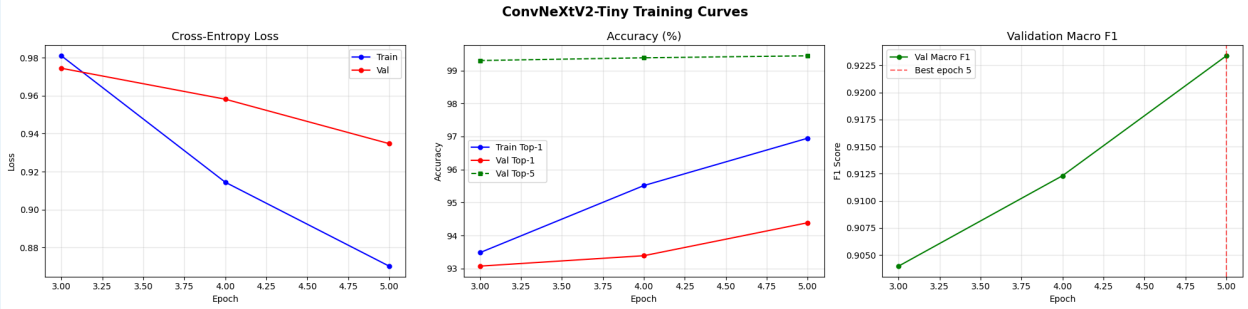


Figure 9: *ConvNeXtV2-Tiny training curves — Loss, Accuracy, and Val Macro F1 per epoch*

5.1.2 EfficientNetV2-S

EfficientNetV2-S provides a strong balance between accuracy and inference speed. It was trained using a two-phase fine-tuning strategy.

Parameter	Value
Parameters	20.3 M (tf_efficientnetv2_s)
Input image size	300 × 300 px
Phase 1 (head-only)	3 epochs — backbone frozen; head LR 1e-3
Phase 2 (full fine-tune)	7 epochs — backbone LR 1e-5, classifier LR 1e-4
Augmentation	MixUp, CutMix (Phase 2); standard transforms (Phase 1)
Normalisation	Mean/std = [0.5, 0.5, 0.5]
Test Top-1 Accuracy	92.85%
Test Macro F1	0.9024
Inference throughput	109.2 images/sec (Tesla T4)

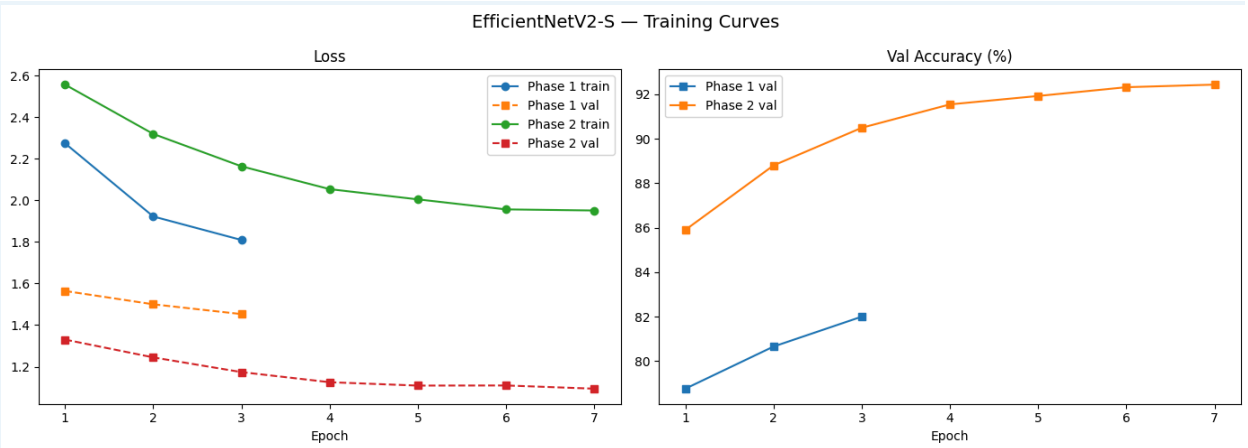


Figure 10: *EfficientNetV2-S training curves — Phase 1 & Phase 2 loss and validation accuracy*

5.2 YOLO Detection Models (Earlier Pipeline)

Two YOLO variants were trained in Milestones 3–4 on the merged detection dataset (118 classes, 39,185 images):

Model	Epochs	Batch	Precision	Recall	mAP@50	mAP@50-95
YOLOv26n	30	8	0.807	0.768	0.840	0.672
YOLOv12s	50	32	0.891	0.877	~0.90	~0.75

YOLOv12s used AdamW optimiser, Mosaic + HSV jitter + random erasing augmentation, and mixed precision training on a Tesla T4 GPU. Both models were superseded by the segmentation-first pipeline in the final architecture.

5.3 MobileNetV2 (Baseline Reference)

MobileNetV2 was trained as a lightweight baseline for initial validation of the classification approach. With a 128×128 input, 10 epochs, Adam optimiser, and CPU training, it achieved 72.06% accuracy and 0.6883 F1-score — sufficient to validate the approach but significantly below the final models.

6. Weight Estimation — From MLP to PCA + Geometry

6.1 Why Depth Estimation Was Rejected

Approach	Suitable?	Reason
ZoeDepth / MiDaS (3-D depth)	No	Dal, sambar, curry are liquid — depth map unreliable
Volume reconstruction	No	Roti/dosa are flat; near-zero height gives incorrect volume
Pixel ratio → MLP	Partial	Camera-distance-dependent; requires labelled regression data
PCA + coin calibration (chosen)	Yes	Geometric, interpretable, no specialised hardware, works for flat and 3-D food shapes

6.2 MLP Regression Approach (Milestones 2–3)

A shallow MLP was trained on 63 custom-photographed plate images. Features: pixel_ratio (food area / plate area), pixel_food, pixel_plate, class_encoded. The MLP directly predicted weight in grams. This approach was abandoned due to its small training set and sensitivity to camera distance.

6.3 Final PCA-Based Geometric Estimation (Milestones 4–5)

The PCA module applies Principal Component Analysis to the foreground pixels of each SAM3 segmentation mask to extract the major and minor axes, providing rotation-invariant estimates of food item height and width.

- PCA identifies the principal axes of the pixel distribution, yielding major axis (height) and minor axis (width).
- A Rs 10 coin (known diameter: 2.7 cm) serves as the calibration reference — detected in the image to compute pixels-per-cm.
- Real-world dimensions are computed by applying the pixel-to-cm scale factor to PCA outputs.
- Volume is estimated by geometric approximation (cylinder for round items, cuboid for flat breads) with assumed thickness per food category.
- Weight (g) = Volume (cm^3) × food-specific density (g/cm^3) from public references.

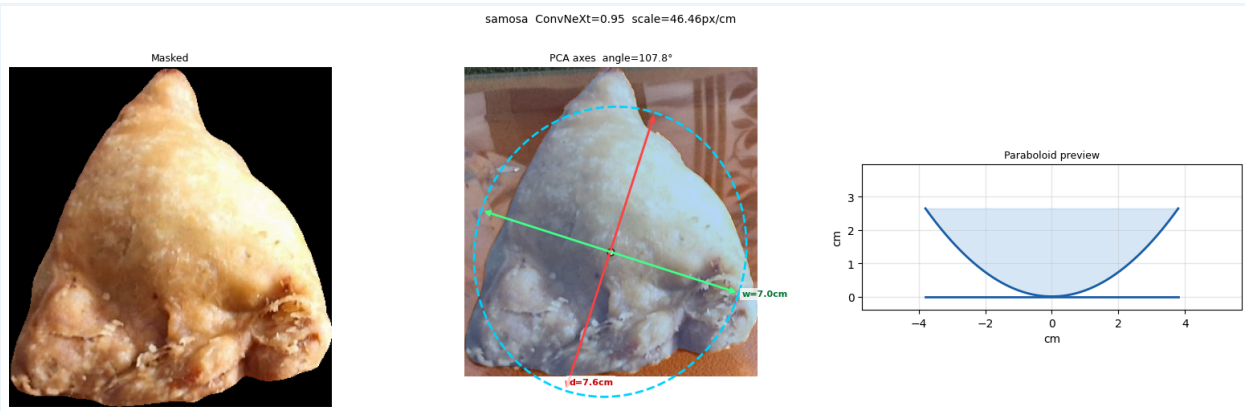
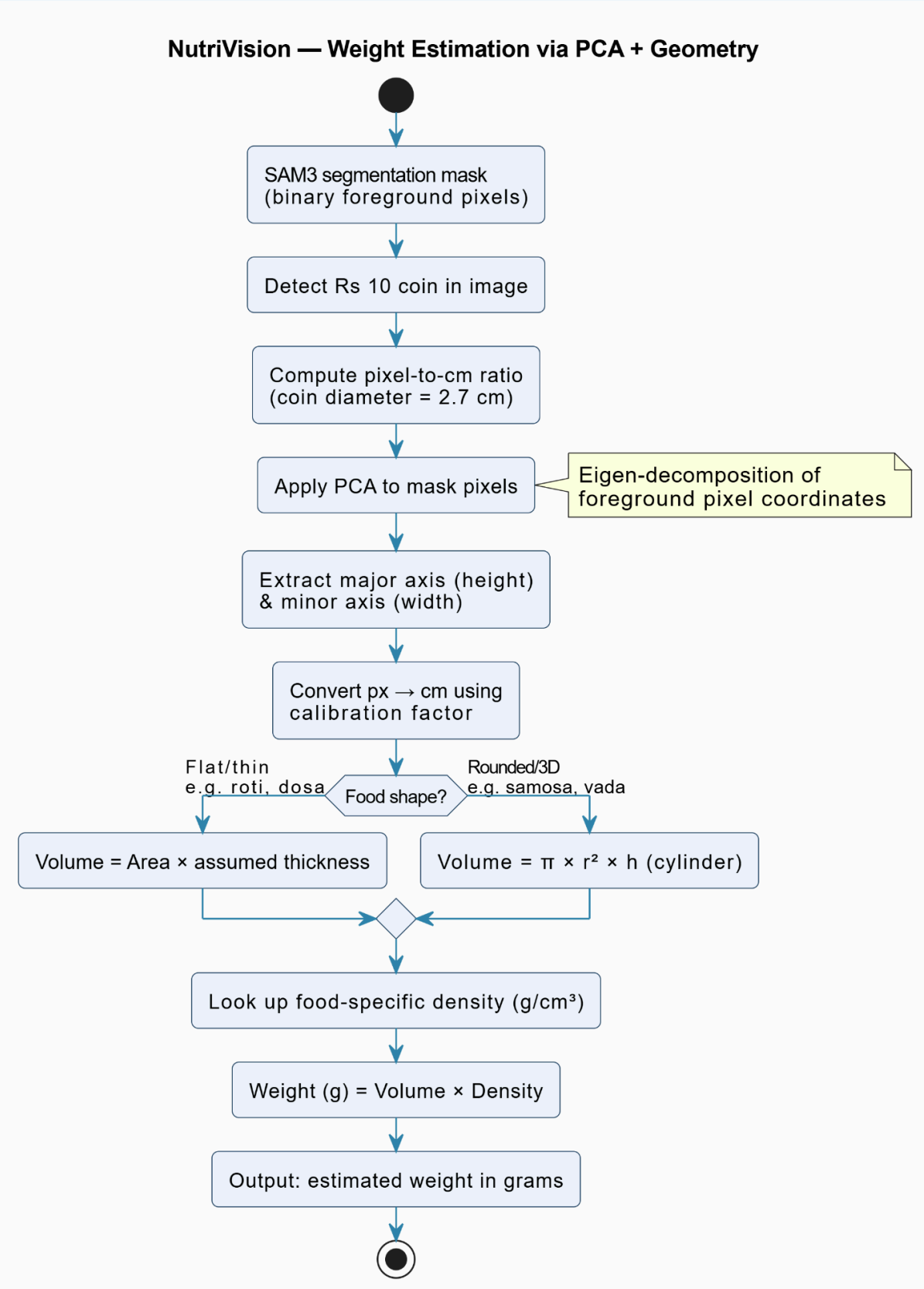


Figure 11: *PCA output on samosa — principal axes overlaid on segmentation mask, plus parabolic profile curve*


NutriVision — Weight Estimation via PCA + Geometry
SAM3 segmentation mask (binary foreground pixels)
Detect Rs 10 coin in image
Compute pixel-to-cm ratio (coin diameter = 2.7 cm)
Apply PCA to mask pixels
Eigen-decomposition of foreground pixel coordinates
Extract major axis (height) & minor axis (width)
Convert px → cm using calibration factor
Flat/thin e.g. roti, dosa
Food shape?
Rounded/3D e.g. samosa, vada
Volume = Area × assumed thickness
Volume = π × r² × h (cylinder)
Look up food-specific density (g/cm³)
Weight (g) = Volume × Density
Output: estimated weight in grams

7. Model Evaluation & Results

All classification models were evaluated on the stratified 10% hold-out test set (13,182 images) using standardised per-model preprocessing. Evaluation was performed in a reproducible Colab notebook with a fixed random seed of 42.

7.1 Classification Results Summary

Model	Top-1 Acc	Top-5 Acc	Precision	Recall	Macro F1	mAP
EfficientNetV2-S	92.85 %	99.12%	0.9251	0.8877	0.9024	0.9387
ConvNeXtV2-Tiny	97.92 %	99.76%	0.9746	0.9753	0.9746	0.9859
Ensemble (avg)	**98.00 %**	99.75%	0.9771	0.9757	0.9761	**0.9860**

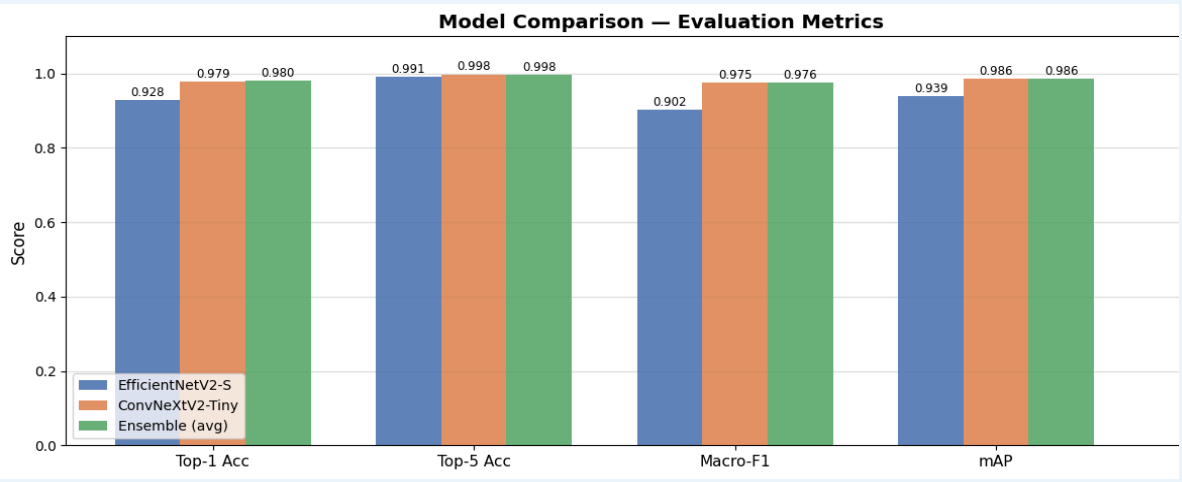


Figure 12: *Model Comparison Bar Chart — Top-1, Top-5, Macro-F1, mAP for all three configurations*

7.2 Confusion Matrix Analysis

Both models produce a strong diagonal pattern in their 80-class confusion matrices, indicating accurate predictions across the majority of categories. The dominant confusion pattern in both models is between visually similar pizza variants:

Confused Pair	EfficientNetV2-S	ConvNeXtV2-Tiny
Paneer pizza → Chicken pizza	48 cases	24 cases
Chicken pizza → Paneer pizza	38 cases	11 cases
Chicken pizza → Margherita pizza	20 cases	9 cases
Phirni / Kheer	Some cases	Negligible

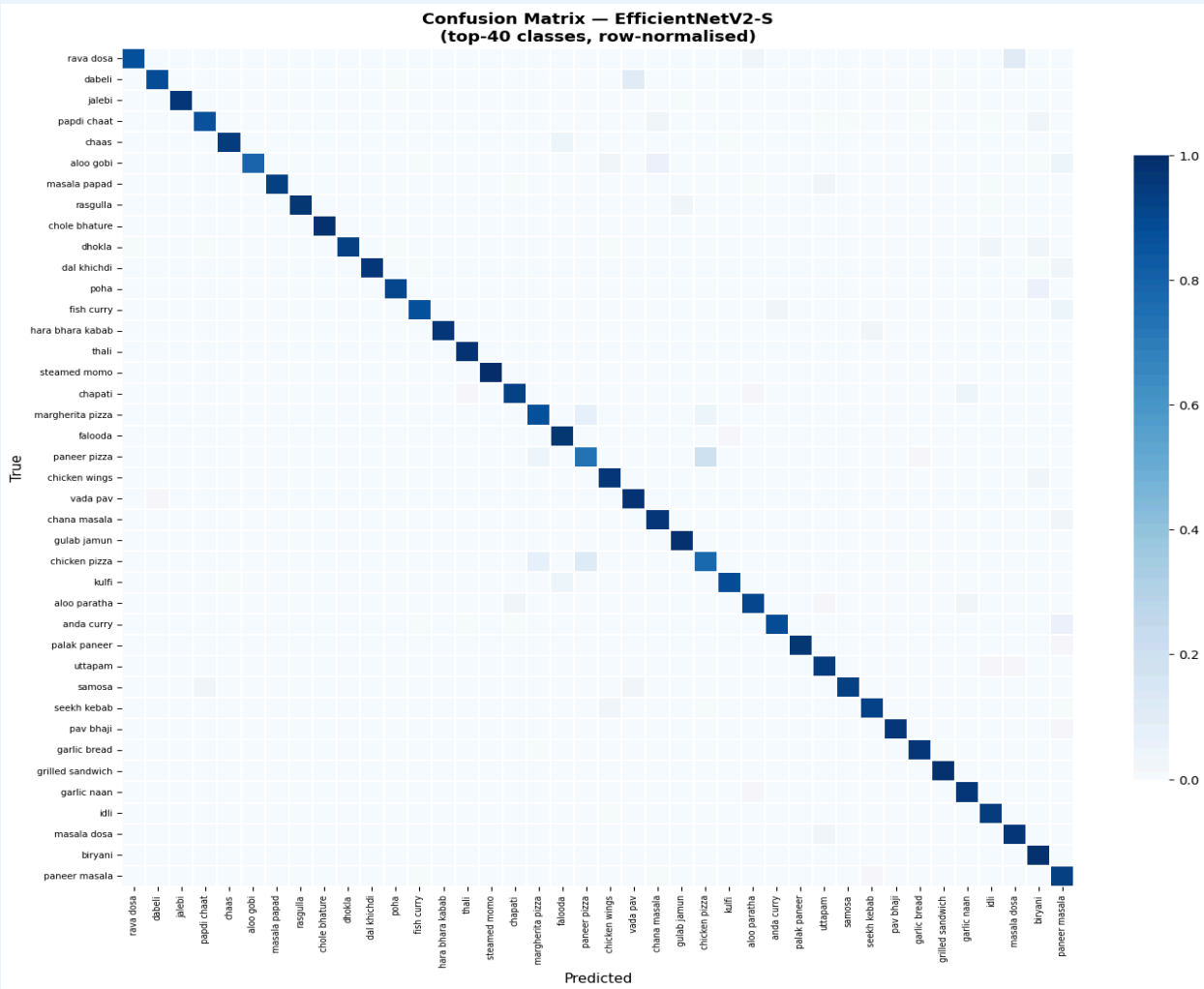


Figure 13: *Confusion matrix — EfficientNetV2-S (top 40 classes, row-normalised)* ▸ Insert from: **milestone5_evaluation_FIXED__5_.ipynb** *[EfficientNetV2-S confusion matrix cell (Milestone 5, Section 6.1.1)]*

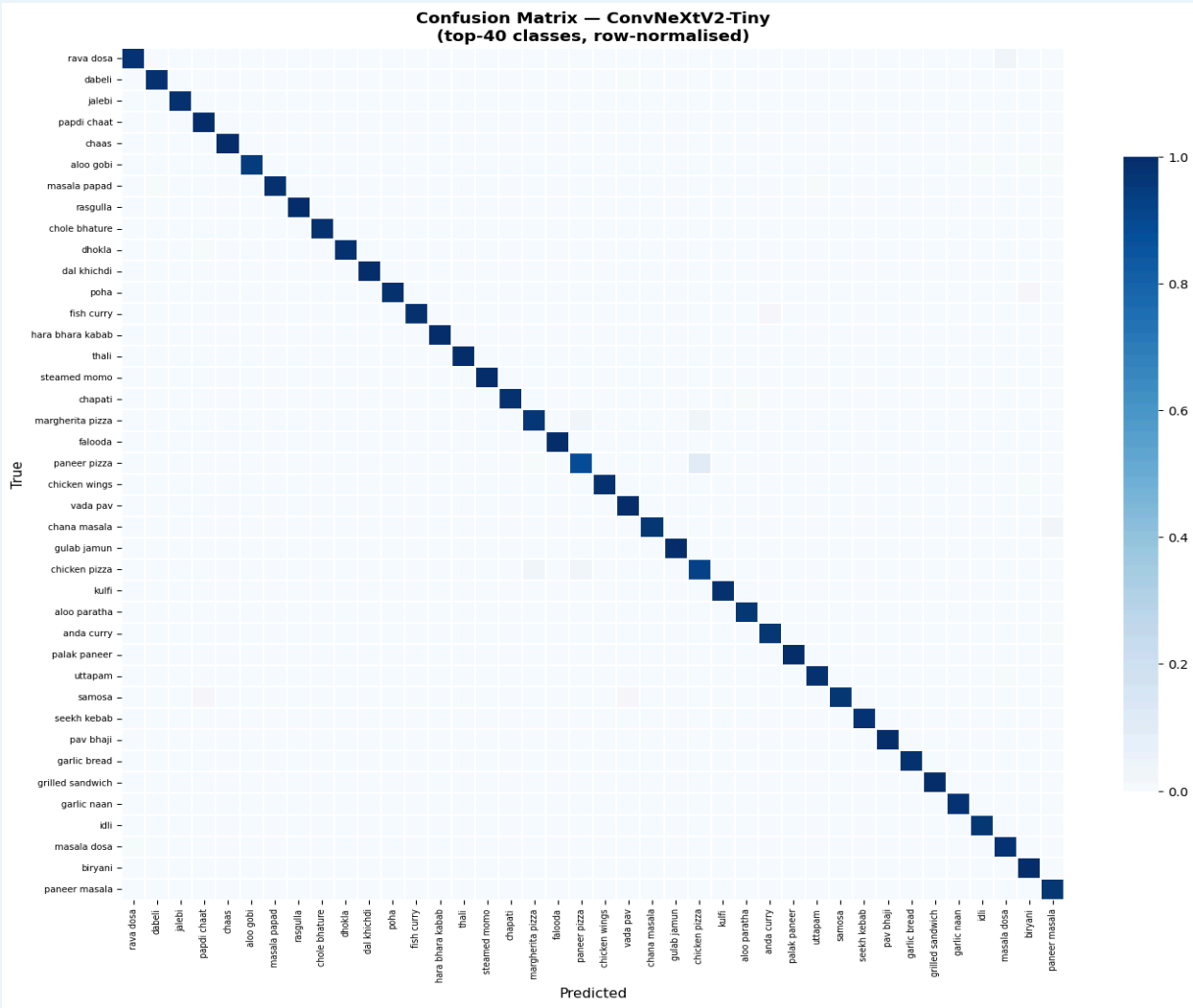


Figure 14: *Confusion matrix — ConvNeXtV2-Tiny (top 40 classes, row-normalised)*

7.3 ROC-AUC & Precision-Recall Curves

Micro-averaged ROC curves show both models well above the random baseline. EfficientNetV2-S achieves micro AUC = 0.999; ConvNeXtV2-Tiny and the ensemble achieve micro AUC ≈ 1.000. Macro-averaged AUC returned NaN due to numerical edge cases with 80 imbalanced classes and was therefore excluded from the summary table. Micro-averaged precision-recall AP scores: EfficientNetV2-S = 0.970, ConvNeXtV2-Tiny = 0.992, Ensemble = 0.992.

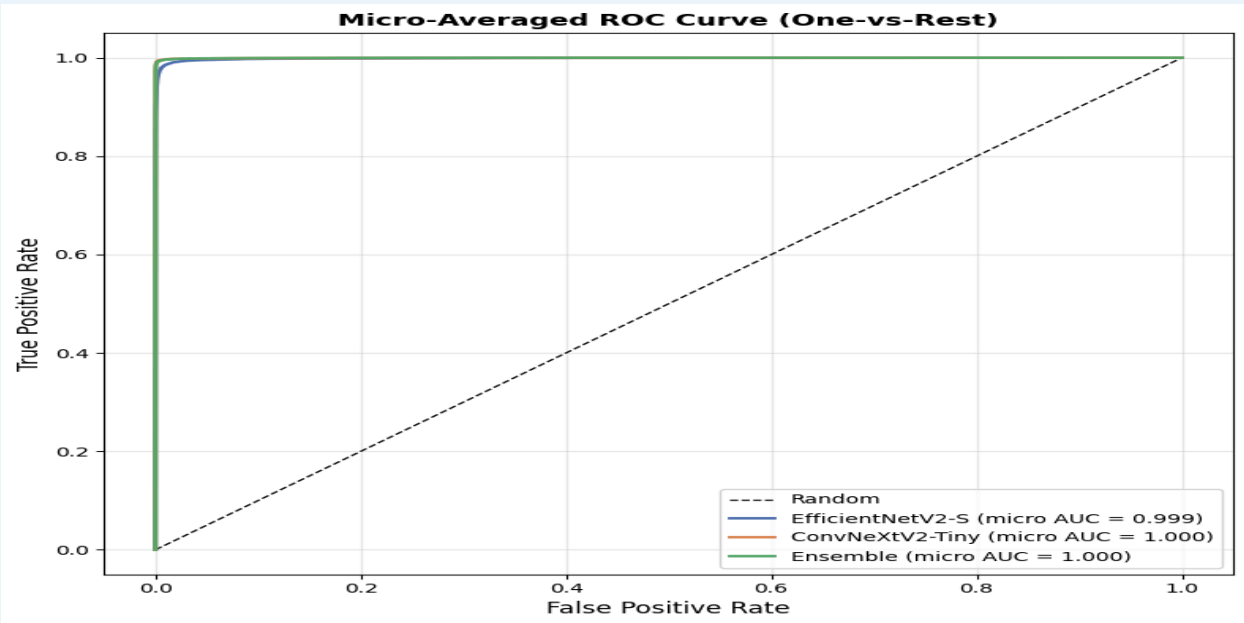


Figure 15: *Micro-averaged ROC curves for all three model configurations*

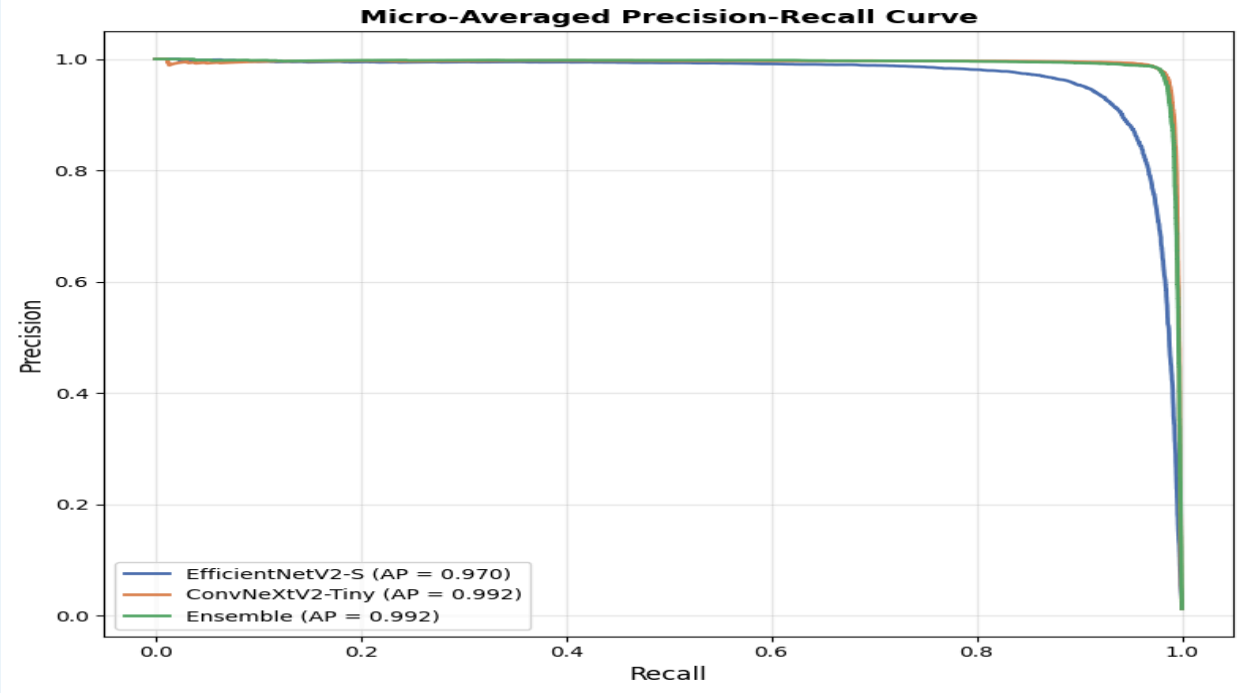


Figure 16: *Micro-averaged Precision-Recall curves for all three model configurations*

7.4 Per-Class Performance

EfficientNetV2-S — strongest classes: steamed momo (F1=0.994), sabudana vada (0.989), grilled sandwich (0.983). Weakest: amritsari kulcha (0.486), phirni (0.645), moong dal halwa (0.686). ConvNeXtV2-Tiny achieves uniformly high scores; fewer than a handful of classes fall below F1=0.90, all involving pizza variants or very low-sample classes.

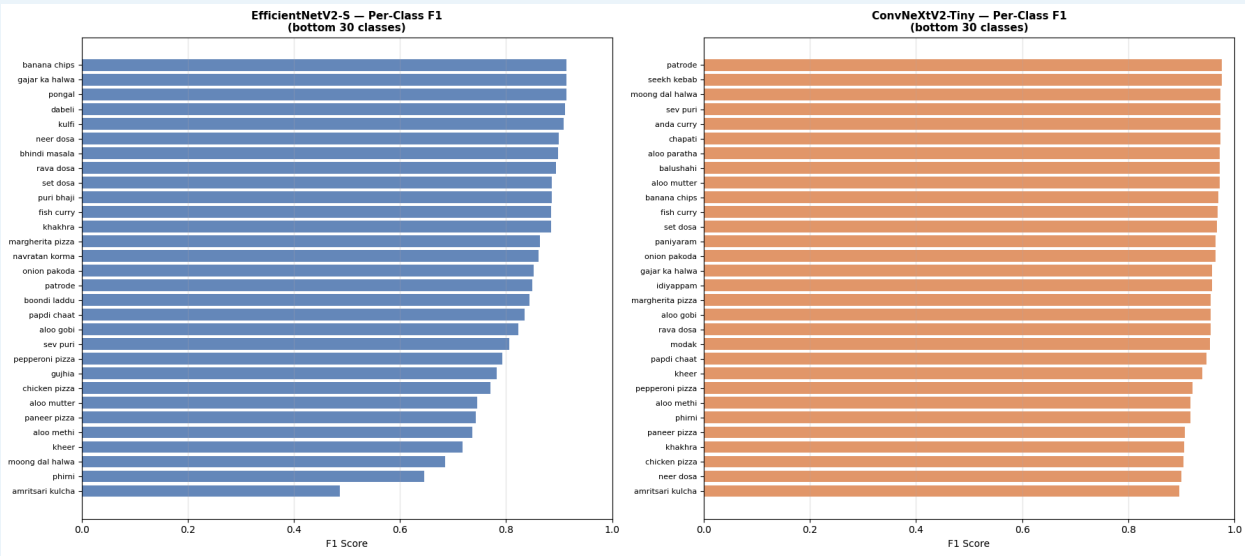


Figure 17: *Per-class F1 bar charts — bottom 30 classes for EfficientNetV2-S and ConvNeXtV2-Tiny*

7.5 Throughput & Latency

Model	Params (M)	Image Size	Inf Time (s)	Images/sec
EfficientNetV2-S	20.3	300 px	120.8 s / 13K imgs	109.2
ConvNeXtV2-Tiny	27.9	176 px	87.5 s / 13K imgs	150.7

8. Generalisation & Optimisation Techniques

Multiple techniques were combined across the classification training pipelines to improve generalisation and training stability:

8.1 Two-Phase Fine-Tuning (EfficientNetV2-S)

Phase 1 froze the backbone for 3 epochs, training only the classification head at high LR to avoid corrupting pretrained representations. Phase 2 unfroze the entire network with a 100× lower backbone LR, allowing gradual adaptation. This strategy prevents catastrophic forgetting on the 80-class Indian food task.

8.2 Cosine LR Scheduling with Warmup

Both models used 1-epoch linear warmup followed by cosine decay. Warmup prevents large gradient updates that destabilise pretrained weights in the first epoch; cosine decay ensures smooth convergence without oscillation.

8.3 Label Smoothing

Cross-entropy loss with label smoothing ε = 0.1 distributed a small probability (0.1 / 80) to all non-target classes. This prevents the model from becoming overconfident and improves calibration — especially important for the 80-class setting where inter-class similarity is high.

8.4 Weighted Random Sampling

The Khana dataset has extreme class imbalance (max 7,000 vs min <20 images per class). ConvNeXtV2's training used a weighted random sampler to ensure each class was seen proportionally, preventing bias toward high-frequency dishes like biryani.

8.5 Mixed Precision Training (AMP)

Both models used torch.amp.autocast for float16 forward passes with float32 gradient accumulation, reducing GPU memory by ~40–50% and accelerating training on Tesla T4 Tensor Cores.

8.6 Gradient Accumulation

A true batch size of 256 was infeasible on the T4. Gradient accumulation (4 × physical batch of 64) simulated effective batch 256 without out-of-memory risk. This was the single highest-impact hyperparameter intervention in the grid search (+3.44 Val F1 points over the batch-64 baseline).

8.7 MixUp and CutMix (EfficientNetV2-S)

Applied during Phase 2 full fine-tuning. MixUp linearly interpolates two training images and their labels; CutMix pastes a rectangular crop from one image into another. Both encourage soft predictions and improve calibration without changing the dataset size.

8.8 Stochastic Depth (EfficientNetV2-S)

Randomly drops residual branches during training, acting as an implicit ensemble over sub-networks and reducing overfitting in deep networks.

9. Nutrition Retrieval & LLM Recommendation Pipeline

9.1 Nutrition Lookup

After weight estimation, nutritional values are retrieved from the structured Indian food CSV database. Per-100g values for calories, protein, carbohydrates, fat, fibre, sodium, and iron are scaled by the estimated portion weight. For a detected item of weight W grams, each nutrient value N (per 100g) becomes: Actual = N × (W / 100). All detected food items on the plate are aggregated to produce the full meal's nutritional profile.

9.2 Daily Tracking vs. ICMR RDA

Cumulative daily intake is compared against ICMR Recommended Dietary Allowances specific to the user's age, gender, and activity level. ICMR-NIN 2024 guidelines are used as the reference. Selected RDA values:

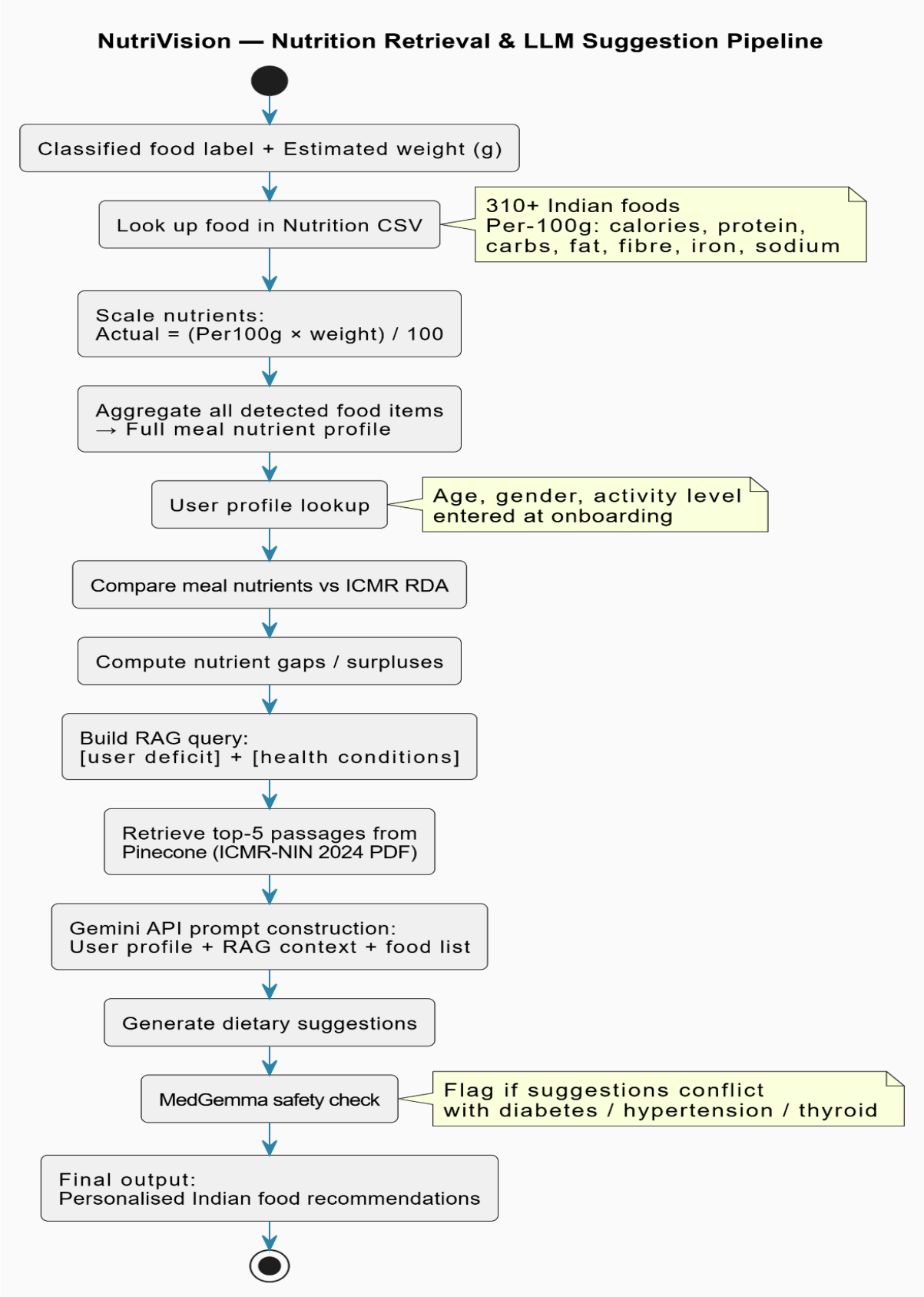
User Profile	Calories (kcal)	Protein (g)	Carbs (g)	Fat (g)	Iron (mg)
Male 19-29, Sedentary	2110	54	304	58	17
Male 19-29, Moderate	2710	54	390	75	17
Female 19-29, Sedentary	1660	46	239	46	29
Female 19-29, Moderate	2110	46	304	58	29

9.3 RAG Pipeline — Pinecone + ICMR Documents

ICMR Dietary Guidelines for Indians 2024 are split into 512-token chunks and embedded using Google text-embedding-004. Embeddings are stored in a Pinecone serverless index. At inference, the user's nutrient deficit and health profile are sent as a query; the top 5 retrieved passages are injected as context into the Gemini API prompt.

9.4 Gemini API + MedGemma — Suggestion Generation

Model	Role	Contribution
Gemini API	Primary reasoning	Reads user profile + RAG context + Indian food list → specific portion recommendations
MedGemma	Medical safety layer	Checks Gemini output for conflicts with user health conditions (diabetes, hypertension, thyroid)


NutriVision — Nutrition Retrieval & LLM Suggestion Pipeline
Classified food label + Estimated weight (g)
Look up food in Nutrition CSV
310+ Indian foods
Per-100g: calories, protein, carbs, fat, fibre, iron, sodium
Scale nutrients:
Actual = (Per100g × weight) / 100
Aggregate all detected food items
→ Full meal nutrient profile
User profile lookup
Age, gender, activity level entered at onboarding
Compare meal nutrients vs ICMR RDA
Compute nutrient gaps / surpluses
Build RAG query:
[user deficit] + [health conditions]
Retrieve top-5 passages from Pinecone (ICMR-NIN 2024 PDF)
Gemini API prompt construction:
User profile + RAG context + food list
Generate dietary suggestions
MedGemma safety check
Flag if suggestions conflict with diabetes / hypertension / thyroid
Final output:
Personalised Indian food recommendations

10. Challenges Faced & How They Were Resolved

Challenge	Resolution
C1 — Inconsistent class names across 3 detection datasets	Canonical mapping strategy normalised all labels; class index remapping applied before merge.
C2 — Severe class imbalance in Khana dataset (max 7,000 vs min <20 images)	Weighted random sampler (ConvNeXtV2) and class-weighted loss (EfficientNetV2). Low-frequency detection classes removed with <50 annotation threshold.
C3 — ConvNeXtV2-Tiny appeared to have collapsed to a single class (M4 bug)	Root cause: evaluation preprocessing used wrong image size (224 instead of 176) and incorrect normalisation statistics. Corrected in M5 with per-model preprocessing — accuracy jumped from near-random to 97.92%.
C4 — MLP weight estimation: too small a dataset, camera-distance sensitive	Redesigned to PCA + coin calibration + density-based geometric volume estimation, eliminating the need for labelled regression data.
C5 — GPU memory constraints for large batch sizes	Gradient accumulation (4 × 64 = effective 256) combined with mixed precision training enabled large-batch dynamics within T4 VRAM budget.
C6 — Pizza-variant confusion (chicken/paneer/margherita)	Fundamental visual similarity — dishes share crust shape, colour, texture. Identified as irreducible with current single-view 2D data. Flagged for future: ingredient-level annotations or multi-view input.
C7 — Macro ROC-AUC returning NaN for 80-class evaluation	Numerical edge cases with imbalanced 80-class test set. Reported micro-averaged AUC instead (0.999 and 1.000), which accurately reflects performance.
C8 — Transition from YOLO+SAM-2 to SAM3 segmentation-first	Motivated by overlapping multi-dish failures of detection-first approach. SAM3 generates masks directly, eliminating dependency on accurate bounding boxes.

12. Key Findings, Limitations & Future Work

12.1 Key Findings

- ConvNeXtV2-Tiny is the strongest single classifier: Top-1 97.92%, Macro F1 0.9746, mAP 0.9859 — outperforming EfficientNetV2-S by ~5% accuracy on this dataset.
- The ensemble marginally improves over ConvNeXtV2-Tiny alone (+0.08% Top-1) because both models make complementary errors on only a small fraction of images.
- Effective batch size (via gradient accumulation) is the most impactful hyperparameter: +3.44 Val F1 points over batch-64 baseline.
- Conservative backbone LR is critical: reducing from 1e-4 to 5e-5 gave +3.30 points; increasing to 3e-4 caused catastrophic forgetting (-6.57 points).
- Preprocessing bugs caused the most misleading result in the project: incorrect image size and normalisation made ConvNeXtV2-Tiny appear near-random when it was actually the best model.
- The PCA + coin-calibration weight estimation approach is more practical than depth estimation or MLP regression for Indian food images.
- Top-5 accuracy of 99.53–99.76% demonstrates that the models' learned representations are semantically well-organised: the correct class is almost always within the top 5 predictions.

12.2 Limitations

- Pizza-variant confusion (chicken/paneer/margherita) is the dominant error pattern for both models — visually irreducible with 2D images alone.
- Minority classes with fewer than 30 test samples (e.g., aloo methi: 12, neer dosa: 21) show high metric variance and limited reliability.
- Dataset diversity may not cover all real-world photography conditions (overhead shots, partial occlusion, dim lighting, phone-camera lens distortion).
- PCA weight estimation is approximate: accuracy depends on clean segmentation masks, correct food shape assumptions, and absence of garnishes or partial occlusion.
- Macro ROC-AUC numerical issues with 80 imbalanced classes; micro-AUC used as substitute.

12.3 Future Work

- Longer retraining with best hyperparameters (BS_256_GA configuration) for 5+ epochs — expected to further improve ConvNeXtV2-Tiny beyond 97.92%.
- Ingredient-level annotations or multi-view input for pizza-variant disambiguation.
- Real-time depth estimation (ZoeDepth) for improved 3D volume estimation on non-flat dishes.
- Self-supervised pretraining on domain-specific Indian food images before fine-tuning.
- Expanding the dataset to 150+ classes including regional specialties and street foods.

- Mobile deployment optimisation (quantisation, TensorRT) for on-device inference.

13. Conclusion

NutriVision successfully demonstrates a complete, end-to-end AI pipeline for Indian food nutrition estimation. The system evolved significantly across five milestones — from a YOLO-based detection approach to a SAM3 segmentation-first architecture with dedicated ConvNeXtV2/EfficientNetV2 classification backbones and PCA-based geometric weight estimation.

The final classification models achieve exceptional performance: ConvNeXtV2-Tiny at 97.92% Top-1 accuracy and 0.9746 Macro F1 across 80 Indian food classes, with the ensemble reaching 98.00%. These results establish the system as a strong foundation for practical food logging, calorie estimation, and personalised dietary recommendations tailored to Indian cuisine.

The most valuable lessons from the project include: the critical importance of per-model preprocessing in evaluation, the outsized impact of effective batch size and conservative backbone learning rates in fine-tuning, and the practical superiority of geometry-based weight estimation over regression approaches when labelled data is scarce. All components are modular and independently upgradeable, making NutriVision a robust platform for further development.